

PRÁCTICA GUIADA – ETAPA 6

PROYECTO SERVICIOS TÉCNICOS

CRUD de Clientes: registrar, consultar, editar y eliminar

1. Propósito y objetivos

Implementar las cuatro operaciones CRUD (Create, Read, Update, Delete) para Clientes, usando React como interfaz, Flask como API REST y MySQL como base de datos.

Crear registros con POST.
Consultar con GET.
Actualizar con PUT.
Eliminar con DELETE.
Validar entradas, manejar errores y códigos HTTP.
Probar con Thunder Client y desde React.
Entregar evidencias y subir cambios a GitHub.

2. Requisitos previos

Etapa 5 completada.
MySQL, Flask y Vite instalados.
Thunder Client disponible en VS Code.
Base tecnico accesible
.

3. Confirmar el esquema

En MySQL ejecuta:

```
USE tecnico;
SHOW TABLES;
DESCRIBE cliente;
```

Los ejemplos suponen columnas id y nombre. Ajusta los nombres y agrega los campos obligatorios que realmente tenga tu tabla. Trabaja solo con registros de prueba.

4. Iniciar los servidores

En una terminal:

```
cd backend
venv\Scripts\activate
python -m app.main
```



En otra terminal:

```
cd frontend
npm run dev
```

Direcciones locales: API http://127.0.0.1:5000 y frontend http://localhost:5173/.

5. API Flask: imports y Blueprint

En backend/app/routes/clientes.py, conserva el Blueprint y GET de la etapa anterior. Verifica estos imports:

```
from flask import Blueprint, jsonify, request
from config.conexion import conectar
```

6. Endpoint POST para crear

```
@clientes_bp.route("/api/clientes", methods=["POST"])
def crear_cliente():
    datos = request.get_json(silent=True)
    if not isinstance(datos, dict):
        return jsonify({"success": False, "message": "JSON válido requerido"}), 400
    nombre = datos.get("nombre")
    if not isinstance(nombre, str) or not nombre.strip():
        return jsonify({"success": False, "message": "El nombre es obligatorio"}), 400
    conexion = conectar()
    if conexion is None:
        return jsonify({"success": False, "message": "Sin conexión a BD"}), 500
    cursor = None
    try:
        cursor = conexion.cursor()
        cursor.execute("INSERT INTO cliente (nombre) VALUES (%s)", (nombre.strip(),))
        conexion.commit()
        return jsonify({"success": True, "message": "Cliente creado",
                        "id": cursor.lastrowid}), 201
    except Exception as e:
        conexion.rollback()
        return jsonify({"success": False, "message": str(e)}), 500
    finally:
        if cursor: cursor.close()
        conexion.close()
```

Si tu tabla tiene más columnas obligatorias, inclúyelas en la validación, INSERT y JSON.

7. Endpoint PUT para actualizar

```
@clientes_bp.route("/api/clientes/<int:cliente_id>", methods=["PUT"])
def actualizar_cliente(cliente_id):
    datos = request.get_json(silent=True)
    if not isinstance(datos, dict):
        return jsonify({"success": False, "message": "JSON válido requerido"}), 400
    nombre = datos.get("nombre")
```




```
    if not isinstance(nombre, str) or not nombre.strip():
        return jsonify({"success": False, "message": "El nombre es obligatorio"}), 400
    conexion = conectar()
    if conexion is None:
        return jsonify({"success": False, "message": "Sin conexión a BD"}), 500
    cursor = None
    try:
        cursor = conexion.cursor()
        cursor.execute("UPDATE cliente SET nombre=%s WHERE id=%s",
                       (nombre.strip(), cliente_id))
        conexion.commit()
        if cursor.rowcount == 0:
            return jsonify({"success": False, "message": "No encontrado o sin cambios"}), 404
        return jsonify({"success": True, "message": "Cliente actualizado"}), 200
    except Exception as e:
        conexion.rollback()
        return jsonify({"success": False, "message": str(e)}), 500
    finally:
        if cursor: cursor.close()
        conexion.close()
```

8. Endpoint DELETE para eliminar

```
@clientes_bp.route("/api/clientes/<int:cliente_id>", methods=["DELETE"])
def eliminar_cliente(cliente_id):
    conexion = conectar()
    if conexion is None:
        return jsonify({"success": False, "message": "Sin conexión a BD"}), 500
    cursor = None
    try:
        cursor = conexion.cursor()
        cursor.execute("DELETE FROM cliente WHERE id=%s", (cliente_id,))
        conexion.commit()
        if cursor.rowcount == 0:
            return jsonify({"success": False, "message": "Cliente no encontrado"}), 404
        return jsonify({"success": True, "message": "Cliente eliminado"}), 200
    except Exception as e:
        conexion.rollback()
        return jsonify({"success": False, "message": str(e)}), 500
    finally:
        if cursor: cursor.close()
        conexion.close()
```

Las consultas parametrizadas (%s) evitan concatenar directamente valores recibidos. Si hay relaciones entre tablas, una restricción puede impedir borrar el registro; no elimines datos relacionados sin autorización.



9. Registrar Blueprint

Confirma en backend/app/main.py que el Blueprint de clientes esté importado y registrado, como en la etapa 5. Guarda y reinicia Flask con Ctrl+C y luego python -m app.main.

10. Pruebas paso a paso con Thunder Client

Abre Thunder Client en VS Code y selecciona New Request.
POST a http://127.0.0.1:5000/api/clientes; abre Body → JSON.

```
{"nombre":"Cliente de prueba"}
```

Esperado: 201 Created, success true y un ID. Anota ese ID.

GET http://127.0.0.1:5000/api/clientes; confirma que aparece el registro.
PUT http://127.0.0.1:5000/api/clientes/ID; sustituye ID por el valor real. En Body → JSON envía:

```
{"nombre":"Cliente actualizado"}
```

Esperado: 200 OK. Vuelve a consultar con GET y comprueba el cambio.

DELETE http://127.0.0.1:5000/api/clientes/ID usando solo el ID de prueba.
Ejecuta GET y verifica que ya no aparece. Prueba además POST con nombre vacío (esperado 400) y un ID inexistente (esperado 404).

11. Servicio React

En frontend/src/services/api.js conserva obtenerClientes y agrega funciones:

```
const API_URL = "http://127.0.0.1:5000/api/clientes";

export async function crearCliente(cliente) {
  const r = await fetch(API_URL, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(cliente)
  });
  return { ok: r.ok, datos: await r.json() };
}

export async function actualizarCliente(id, cliente) {
  const r = await fetch(`${API_URL}/${id}`, {
    method: "PUT",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(cliente)
  });
  return { ok: r.ok, datos: await r.json() };
}

export async function eliminarCliente(id) {
```




```
  const r = await fetch(`${API_URL}/${id}`, { method: "DELETE" });
  return { ok: r.ok, datos: await r.json() };
}
```

Importa estas funciones en Clientes.jsx. Si aparece Failed to fetch, comprueba que Flask esté activo y que CORS permita el origen local de Vite.

12. Integrar formulario y acciones en Clientes.jsx

Dentro del componente, agrega estados y manejadores como base para tu implementación. Asegúrate de importar useState y las funciones de api.js:

```
const [nombre, setNombre] = useState("");
const [editandoId, setEditandoId] = useState(null);
const [mensaje, setMensaje] = useState("");

const guardar = async (e) => {
  e.preventDefault();
  const r = editandoId
    ? await actualizarCliente(editandoId, { nombre })
    : await crearCliente({ nombre });
  setMensaje(r.datos.message || "Solicitud procesada");
  if (r.ok) {
    setNombre("");
    setEditandoId(null);
    await cargarClientes();
  }
};

const prepararEdicion = (cliente) => {
  setEditandoId(cliente.id);
  setNombre(cliente.nombre);
};

const borrar = async (id) => {
  if (!window.confirm("¿Eliminar este cliente de prueba?")) return;
  const r = await eliminarCliente(id);
  setMensaje(r.datos.message || "Solicitud procesada");
  if (r.ok) await cargarClientes();
};
```

En el JSX crea un formulario con onSubmit={guardar}, un input controlado por value={nombre} y onChange={e => setNombre(e.target.value)}. En cada fila agrega botones Editar y Eliminar que llamen prepararEdicion(cliente) y borrar(cliente.id). Añade Cancelar para limpiar editandoId y nombre. Ajusta cliente.id y cliente.nombre a los campos reales.



13. Comportamiento esperado y códigos HTTP

200: consulta, actualización o eliminación aceptada.
201: registro creado.
400: JSON inválido o campo requerido ausente.
404: identificador no encontrado o ruta inexistente.
500: error del servidor; revisar terminal Flask y MySQL.
Validar nombre no vacío, mostrar mensajes y refrescar la tabla tras operaciones exitosas.
Pedir confirmación antes de borrar; no exponer credenciales de MySQL en React.

14. Solución de problemas

Problema	Qué revisar
405 Method Not Allowed	methods en el decorador y método seleccionado.
400 Bad Request	Body JSON y Content-Type application/json.
404	URL, Blueprint e ID real.
500 SQL	Nombre real de tabla/columnas, permisos y restricciones.
Failed to fetch	Servidor Flask y configuración CORS.
La tabla no cambia	Invocar cargarClientes tras respuesta exitosa.

15. Evidencias de entrega

Captura de SHOW TABLES y DESCRIBE cliente.
Código de endpoints POST, PUT y DELETE.
Thunder Client: POST 201, GET, PUT 200, DELETE 200.
Prueba de validación 400 y caso 404.
Interfaz React mostrando listado, alta, edición y eliminación.
Tabla actualizada después de cada acción.
Repositorio GitHub actualizado.



16. Registro de pruebas

Caso	Método/ruta	Esperado	Resultado	Evidencia
Crear	POST /api/clientes	201		
Listar	GET /api/clientes	200		
Editar	PUT /api/clientes/{id}	200		
Eliminar	DELETE /api/clientes/{id}	200		
Nombre vacío	POST	400		
ID inexistente	DELETE /api/clientes/999999	404		

17. Preguntas de análisis

¿Qué significa CRUD?
Respuesta: _____

¿Qué diferencia existe entre POST y PUT?
Respuesta: _____

¿Por qué se usan consultas parametrizadas?
Respuesta: _____

¿Para qué sirven commit y rollback?
Respuesta: _____

¿Qué significa el código 201?
Respuesta: _____

¿Por qué se solicita confirmación antes de eliminar?
Respuesta: _____

¿Cómo se actualiza la tabla en React?
Respuesta: _____

18. Cierre y siguiente etapa

La etapa se considera completada cuando las cuatro operaciones funcionan con datos de prueba desde Thunder Client y React, los errores básicos se controlan y el repositorio está actualizado.